Integración por Monte Carlo en memoria compartida con OpenMP

Integrante:

David Steven Anaya González - 2211714

Introducción a la Programación Paralela

Universidad Industrial de Santander

2026-2

Parte I. Fundamento del método

La integración por Monte Carlo en pocas palabras

La integración por Monte Carlo estima el valor de una integral mediante muestreo aleatorio. Para aproximar

$$I = \int_a^b f(x)\,dx,$$

se generan N puntos $x_1, x_2, \ldots, x_N$ distribuidos de manera uniforme en $[a, b]$ y se calcula el estimador

$$I_N = \frac{b-a}{N} \sum_{i=1}^{N} f(x_i).$$

Por la ley de los grandes números, I_N converge al valor exacto de la integral cuando N crece; el error estadístico típico disminuye en proporción a $1/\sqrt{N}$, de modo que cuadruplicar el número de muestras reduce el error a la mitad.
Un caso particular le servirá en esta evaluación: el área del cuarto de círculo unitario es $\pi/4$, es decir,

$$\int_0^1 \sqrt{1-x^2}\,dx = \frac{\pi}{4}.$$

En la variante de *acierto y fallo*, se generan puntos (x, y) uniformes en el cuadrado $[0, 1] \times [0, 1]$ y se cuenta la fracción que cae dentro del cuarto de círculo $x^2 + y^2 \leq 1$; esa fracción multiplicada por 4 aproxima π. Cada muestra es independiente de las demás, y lo único que se acumula a lo largo del cálculo es el número de aciertos.

Preguntas

I.1. Explique con sus propias palabras por qué el error del estimador disminuye en proporción a 1/ √ N y deduzca cuántas muestras adicionales se requieren para reducir el error a la mitad. ¿Qué implicación tiene esto sobre el costo de ganar un dígito de precisión?

Para hacer explicación del error del estimador, es necesario recapitular la información proporcionada anteriormente. Teniendo en cuenta lo ya mencionado, se ha optado por usar la variante de acierto y fallo que se explicó en "*La integración por Monte Carlo en pocas palabras*".

Cito: “En la variante de acierto y fallo, se generan puntos (x,y) uniformes en el cuadrado [0,1] x [0,1] y se cuenta la fracción que cae dentro del cuarto de círculo $x^2+y^2 <= 1$; esta fracción multiplicada por 4 aproxima π .” Ok, entonces tenemos lo siguiente:

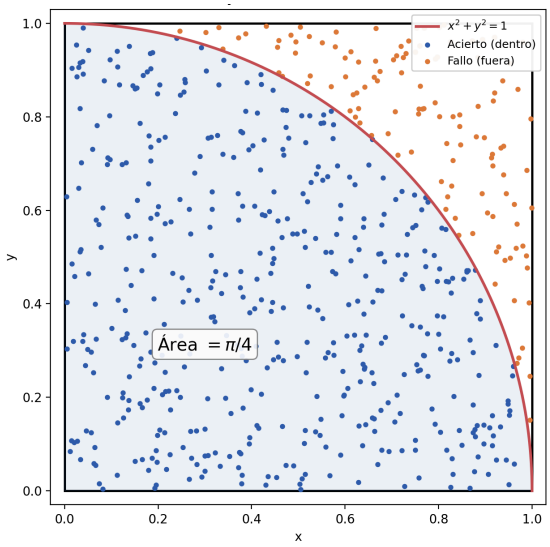


Figura 1. Imagen representativa de lo planteado por la variante acierto y fallo

A sabiendas que el área del cuarto del círculo es π/4 (véase el resultado de la integral) y la de un cuadrado está dada por L^2, siendo su valor igual a 1. Si se tiene en cuenta que los puntos están distribuidos de manera uniforme en el cuadrado, entonces la proporción de puntos que cae dentro del cuarto de círculo tiende a ser igual a la proporción del área que ocupa.

Por tanto:

$$\frac{Área\ del\ Cuarto\ de\ Círculo}{Área\ del\ cuadrado} \approx \frac{Puntos\ dentro\ del\ círculo}{Puntos\ totales}$$

Sustituimos valores y simplificamos

$$\frac{\pi}{4} \approx \frac{Puntos\ dentro\ del\ círculo}{Puntos\ totales}$$

Para eliminar la fracción y dejar el aproximado de π , entonces multiplicamos por 4 en ambos lados:

$$\pi \approx 4\frac{Puntos\ dentro\ del\ círculo\ (M)}{Puntos\ totales\ (N)}$$

Ahora, tomemos que la proporción de aciertos como un estimador probabilístico, $\hat{p} = \frac{M}{N}$. Dónde, si tenemos en cuenta el proceso anterior tenemos que $\frac{\pi}{4} \approx \hat{p}$. Pero, ahora me gustaría saber qué tanto se equivoca esta estimación. Para eso modelo cada punto como un ensayo de Bernoulli independiente, como una moneda que cae “acierto” o “fallo” :

$$X_i = \begin{cases} 1, & \text{si el punto } (x_i, y_i) \text{ cae dentro del círculo } (x_i^2 + y_i^2 \leq 1) \\ 0, & \text{en caso contrario} \end{cases}$$

Como los puntos son uniformes en un cuadrado de área 1, la probabilidad de acierto es el área del cuarto de círculo. Aquí llamo p a esa probabilidad teórica, que no es lo mismo que p̂, la proporción que se observa en una corrida:

$$P(X_i = 1) = p = \frac{\pi}{4}$$

Al ser Bernoulli, cada X_i tiene esperanza y varianza conocidas:

- $E[X_i] = p = \frac{\pi}{4}$
- $Var(X_i) = \sigma^2 = p(1 - p) = \frac{\pi}{4}\left(1 - \frac{\pi}{4}\right)$

Mi proporción observada es el promedio de los X_i: $\hat{p} = \frac{1}{N}\sum_{i=1}^{N} X_i$. Como cada punto se lanza de forma independiente de los demás, la varianza de la suma es la suma de las varianzas:

$$\mathrm{Var}(\hat{p}) = \mathrm{Var}\left(\frac{1}{N}\sum_{i=1}^{N} X_i\right) = \frac{1}{N^2}\sum_{i=1}^{N}\mathrm{Var}(X_i) = \frac{N \;\; p(1-p)}{N^2} = \frac{p(1-p)}{N}$$

Dado que la estimación final de π es $\hat{\pi} = 4\hat{p}$, su varianza y su desviación estándar (error típico estadístico $Error(N)$) son:

$$Var(\hat{\pi}) = 16 \cdot Var(\hat{p}) = \frac{16p(1-p)}{N}$$

$$Error(N) = \sqrt{Var(\hat{\pi})} = \frac{4\sqrt{p(1-p)}}{\sqrt{N}} \propto \frac{1}{\sqrt{N}}$$

Ahora, queremos determinar un nuevo valor de N que permita reducir el valor del error a la mitad. Entonces, sencillamente hacemos este proceso:

$$Error(N(new) = \frac{1}{2\sqrt{N}}$$

$$\frac{1}{\sqrt{N(new)}} = \frac{1}{2\sqrt{N}}$$

Despejamos el valor N(new)

$$\sqrt{N(new)} = 2\sqrt{N}$$

$$(\sqrt{N(new)})^2 = (2\sqrt{N})^2$$

$$N(new) = 4N$$

Teniendo en cuenta el dato anterior, sencillamente se hace un diferencia entre el valor nuevo de N con el valor anterior para obtener las muestras adicionales necesarias.

$$\Delta N = 4N - N$$

$$\Delta N = 3N$$

Por lo tanto **la cantidad de muestras adicionales para reducir el error a la mitad es de 3N**, dónde N es la cantidad de muestras totales.

Pero si se busca ganar **un dígito de precisión decimal** en la estimación de π equivale a reducir el error estadístico por un factor de 10 (es decir, conseguir un error de $\frac{1}{10}Error(N)$), seguimos la misma lógica anterior:

$$\frac{1}{\sqrt{N_{\text{nuevo}}}} = \frac{1}{10} \cdot \frac{1}{\sqrt{N}} \implies \sqrt{N_{\text{nuevo}}} = 10\sqrt{N} \implies N_{\text{nuevo}} = 100N$$

Entiéndase que el costo computacional en Monte Carlo escala de forma cuadrática con respecto a la precisión obtenida. Teniendo en cuenta el proceso anterior, para reducir un dígito, se requiere aumentar el volumen de cómputo por un factor de 100. Por lo tanto, el método resultaría útil para obtener aproximaciones rápidas ya sea de baja o mediana precisión, pero se vuelve sumamente costoso cuando se exige precisiones numéricas muy elevadas de forma secuencial.

I.2. Justifique por qué la integración por Monte Carlo se clasifica como un problema embarazosamente paralelo: analice las dependencias entre las muestras (o su ausencia) y determine qué datos deben ser compartidos y cuáles deben ser privados cuando el muestreo se reparte entre varios hilos de ejecución.

La integración por Monte Carlo resulta ser un problema embarazosamente paralelo porque el proceso principal consiste en generar puntos aleatorios e independientes para evaluar condiciones geométricas sin que exista cómo tal una dependencia o comunicación intermedia entre las muestras. Véase en la variante de acierto y fallo, dónde se generan puntos uniformes (x,y) en el cuadrado [0,1] x [0,1] para contar cuántos caen dentro del cuarto de círculo definido por $x^2+y^2 \leq 1$. Si lo miras bien, cada muestra es

completamente independiente de las demás y lo único que cómo tal se acumula a lo largo de todo el cálculo es el número de aciertos, por lo que el problema puede dividirse en múltiples hilos de ejecución sin necesidad de sincronización en la fase de muestreo.

Pero, al repartir este proceso en un entorno de memoria compartida utilizando OpenMP, es necesario definir con precisión el ámbito de cada variable para evitar las condiciones de carrera y garantizar la reproducibilidad. Por el lado de las coordenadas temporales de los puntos (x,y), los índices de iteración y el estado interno del generador de números pseudoaleatorios deben configurarse como datos privados por hilo. Esto resulta indispensable para que se asegure que cada hilo disponga de un flujo de números aleatorios independiente y sin solapamiento, utilizando funciones como ***rand_r(&estado)*** con semillas espaciadas adecuadamente a partir de la semilla base asignada.

Por otro lado, los parámetros de lectura global como el número total de muestras (N) son compartidos, mientras que el acumulador global de aciertos se gestiona mediante una cláusula de reducción adecuada (***reduction*** de OpenMP). Esta cláusula permite que cada hilo acumule sus aciertos en contadores locales independientes durante el ciclo paralelo, combinándolos de forma segura y atómica al finalizar la ejecución para consolidar el resultado total sin corromper los datos.

I.3. Demuestre que repartir las N muestras entre p hilos y sumar los aciertos parciales de cada uno produce el mismo estimador que la versión secuencial. Con el mismo N total, ¿qué precisión estadística cabe esperar respecto de la ejecución con un solo hilo? Justifique.

Para demostrar que ambos enfoques producen el mismo estimador, es necesario tener en cuenta de cómo se calcula la estimación de π en la variante de acierto y fallo. Por el lado de la versión secuencial se procesan N puntos, y a cada punto m se le asigna un valor que es 1 si cae dentro del cuarto de círculo ($x_m^2 + y_m^2 \leq 1$) y 0 en caso contrario. El número total de aciertos es $C_{secuencial} = \sum_{m=1}^{N} c_m$, y la estimación de π es:

$$I_{N,secuencial} = \frac{4}{N} \sum_{m=1}^{N} c_m$$

Para la versión paralela, las N muestras se reparten entre p hilos. Cada hilo k procesa un bloque de N_k muestras y calcula su número de aciertos parcial $C_k = \sum_{i=1}^{N_k} c_{k,i}$. Los bloques son disjuntos, porque ninguna muestra la procesan dos hilos, y su unión es el conjunto completo, porque ninguna queda sin procesar. Por eso $\sum_{k=1}^{p} N_k = N$ y cada par (k, i) corresponde exactamente a una muestra m. Al terminar, la reducción suma los parciales:

$$C_{paralelo} = \sum_{k=1}^{p} C_k$$

Como cada muestra se cuenta una sola vez y la suma de enteros es asociativa y conmutativa, reagrupar los términos en sumas parciales no cambia el total. Por tanto $C_{paralelo} = C_{secuencial}$ y, al evaluar las mismas N muestras, se obtiene $I_{N,paralelo} = I_{N,secuencial}$. Repartir el trabajo entre varios hilos cambia la forma de ejecutarlo, pero no el resultado del estimador. Esto vale cuando ambos programas usan los mismos puntos. Si cada hilo usara su propio flujo aleatorio, los puntos serían distintos y los resultados no coincidieron dígito a dígito, aunque los estimadores seguirán siendo equivalentes en distribución, con la misma esperanza y la misma varianza.

En cuanto a la precisión estadística, con el mismo número total de muestras N cabe esperar la misma precisión que con un solo hilo. El error estándar del estimador disminuye como $1/\sqrt{N}$, según lo visto en I.1, y en esa expresión solo interviene el número total de muestras independientes, no la cantidad de hilos. Repartir el trabajo no cambia cuántas muestras hay ni su independencia. En consecuencia, aumentar el número de hilos reduce el tiempo de ejecución, pero no mejora la precisión mientras N permanezca constante. Esta conclusión supone que los flujos de números aleatorios de los hilos son independientes y no se solapan; si estuvieran correlacionados, las muestras efectivas serían menos de N y el error real sería mayor.

Parte II. Implementación en OpenMP

El siguiente programa secuencial servirá de punto de partida. Estúdielo antes de modificarlo: su estructura es la que usted debe conservar en la versión paralela.

```
/* montecarlo.c -- version secuencial de referencia.
  Uso: ./montecarlo <numero de muestras> */
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
        long N = (argc > 1) ? atol(argv[1]) : 100000000L;
        unsigned int estado = 20261018u; /* semilla del generador */
        long dentro = 0;

        for (long i = 0; i < N; i++) {
        double x = rand_r(&estado) / (double)RAND_MAX;
        double y = rand_r(&estado) / (double)RAND_MAX;
        if (x * x + y * y <= 1.0)
        dentro++;
        }

        printf("Estimacion de pi: %.6f\n", 4.0 * dentro / (double)N);
```

```
    return 0;
}
```

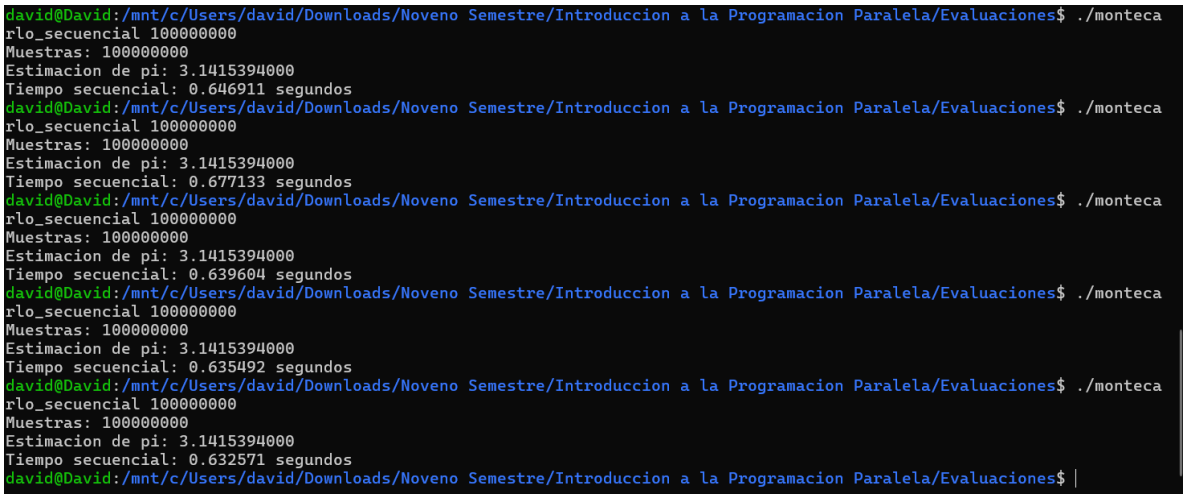
Compílelo con: gcc -O2 montecarlo.c -o montecarlo -lm. Cuando avance a la versión paralela, agregue la bandera -fopenmp; esta también le permitirá medir tiempos con omp_get_wtime(), que necesitará desde la parte II.

Tareas

Dato	Valor en su equipo
Modelo de procesador	11th Gen Intel(R) Core(TM) i5-11300H @ 3.10GHz
Núcleos físicos (Core(s) per socket × Socket(s))	4
Hilos por núcleo	2
Núcleos lógicos (nproc)	8
Valor de _OPENMP	201511

Con semilla 2211714.

II.1 Compile y ejecute la versión secuencial con $N = 10^8$. Verifique que la estimación es consistente con el valor de π (un error del orden de 10^{-4} es normal; explique de dónde sale ese orden de magnitud). Registre el tiempo de ejecución T(1) con omp_get_wtime() y consérvelo para la parte III.



Las cinco ejecuciones dan la misma estimación, 3,1415394. Es lo esperado: la semilla es fija y el generador es determinista, así que el programa recorre siempre la misma secuencia. El número de aciertos es 78 538 485. Frente a $\pi = 3{,}14159265$, el error absoluto es:

$|3{,}1415394 - 3{,}1415927| \approx 5{,}3 \cdot 10^{-5}$ (error relativo $\approx 1{,}7 \cdot 10^{-5}$)

El resultado coincide con el valor de π hasta la cuarta cifra decimal.

¿De dónde sale ese orden de magnitud?

Cada muestra es una prueba de Bernoulli: cae dentro del cuarto de círculo con probabilidad $p = \pi/4 \approx 0{,}7854$. Con N muestras independientes, el número de aciertos tiene desviación estándar $\sqrt{(N \cdot p(1-p))}$. El estimador es $4 \cdot$dentro/N, así que su desviación estándar es:

$$\sigma = 4 \cdot \sqrt{(p(1-p)/N)} = 4 \cdot \sqrt{(0{,}7854 \cdot 0{,}2146 / 10^8)} \approx 1{,}6 \cdot 10^{-4}$$

El error típico tiene el orden de σ, es decir, 10^{-4}. Tu error observado, $5{,}3 \cdot 10^{-5}$, equivale a 0,32σ, algo muy común para una sola realización. Como $\sigma \propto 1/\sqrt{N}$, con $N = 10^8$ se espera precisión de unos 4 dígitos, y ganar un dígito más costaría unas 100 veces más muestras.

II.2 Implemente la versión paralela con OpenMP. Su programa debe:

- **usar una directiva omp parallel for con la cláusula de reducción adecuada sobre el acumulador de aciertos;**
- **permitir elegir el número de hilos p (con omp_set_num_threads o OMP_NUM_THREADS);**
- **garantizar que cada hilo disponga de un flujo de números aleatorios independiente y sin solapamiento con el de los demás. Como semilla base use la que su docente le asignó (las semillas se publican por estudiante) y espacie las de cada hilo al menos N/p valores. El resultado debe ser reproducible: con las mismas semillas, el programa debe producir la misma estimación cualquiera que sea el valor de p.**



La versión paralela distribuye las iteraciones del ciclo entre los hilos mediante omp parallel for. La cláusula reduction(+:dentro) crea acumuladores parciales y suma sus resultados al finalizar la región paralela, evitando que los hilos actualicen simultáneamente un único contador compartido. La planificación schedule(static) reparte las iteraciones de manera predeterminada entre los hilos, con bajo costo de planificación. Para mantener la reproducibilidad al cambiar la cantidad de hilos, se utiliza un generador determinista basado en el índice de la muestra y en la semilla base asignada, 2211714.

II.3 Explique por qué no es correcto que todos los hilos llamen a la función rand() compartiendo el estado global del generador. Su respuesta debe cubrir al menos dos problemas distintos: uno de condición de carrera sobre el estado interno, y otro de correlación entre secuencias si cada hilo inicializara el generador con la misma semilla. Indique qué consecuencias tendría cada uno sobre la estimación de π y sobre el error reportado.

La función rand() utiliza un estado interno que se modifica cada vez que se genera un número aleatorio. Si varios hilos acceden simultáneamente a ese estado sin una sincronización adecuada, pueden presentarse condiciones de carrera, actualizaciones inconsistentes y resultados no deterministas. Incluso cuando una implementación protege internamente el generador, la sincronización puede introducir contención y reducir el rendimiento.

Otro problema aparece si cada hilo inicializa su generador con la misma semilla: los hilos pueden producir secuencias idénticas. En ese caso, se repiten muestras y el número efectivo de observaciones independientes es menor que el esperado. La estimación puede quedar sesgada respecto de la distribución de muestreo prevista y el error estadístico aparente puede ser engañoso.

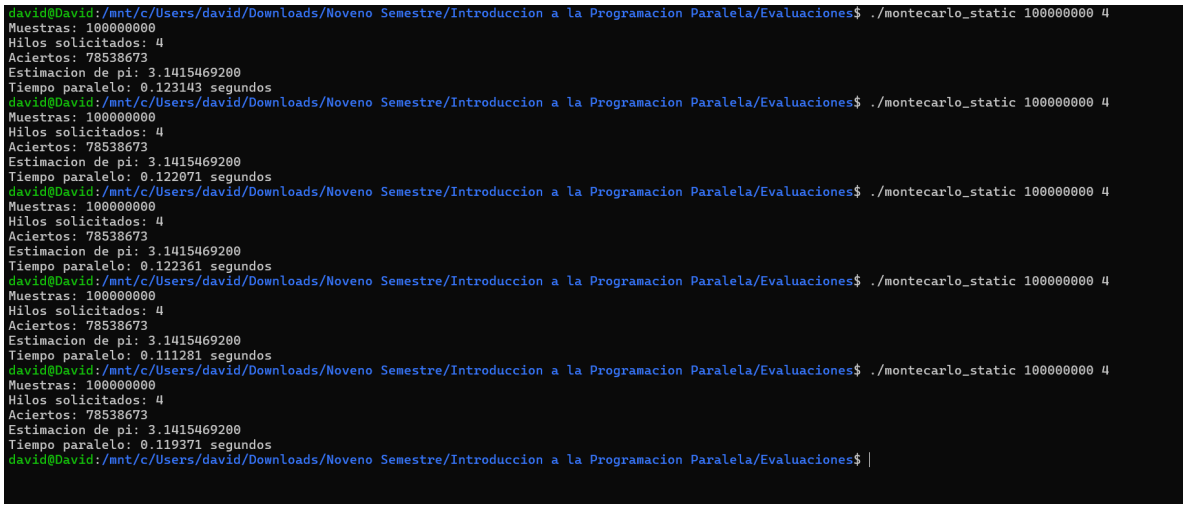
II.4 Analice los atributos de compartición de datos de su región paralela: enumere las variables que OpenMP trata como compartidas y como privadas por defecto en su código, explique la cláusula que usó para el acumulador y describa qué ocurriría si el acumulador se actualizará dentro del ciclo sin esa cláusula (diagnostique el tipo de defecto y su efecto sobre el resultado).

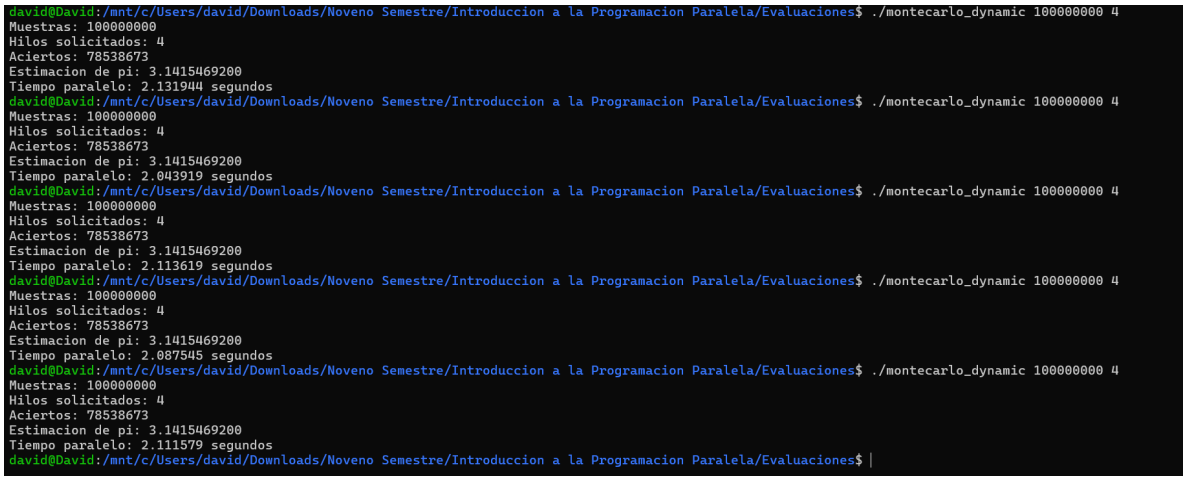
Variable	Tratamiento	Justificación
N	Compartida	Todos los hilos leen el límite del ciclo.

dentro	Reducción	Cada hilo acumula sus aciertos y OpenMP combina los resultados.
i	Privada en el ciclo paralelo	Cada iteración se asigna a un hilo.
indice, x, y	Privadas	Se declaran dentro del ciclo y cada iteración tiene sus propios valores.
SEMILLA_BASE	Compartida, de solo lectura	Es una constante utilizada para calcular las muestras.

Sin **reduction(+:dentro)**, si todos los hilos ejecutarán **dentro++** sobre el mismo acumulador compartido, podría ocurrir una condición de carrera: varios hilos leerían y modificarían el valor simultáneamente, perdiéndose incrementos y obteniéndose una estimación incorrecta.

II.5 (Opcional.) Compare schedule(static) con schedule(dynamic) en su ciclo principal y argumente, con mediciones, si la política de planificación tiene algún efecto observable en este problema y por qué.





Ejecución	static (s)	dynamic (s)
1	0.123143	2.131944
2	0.122071	2.043919
3	0.122361	2.113619
4	0.111281	2.087545
5	0.119371	2.111579
Mediana	0.122071	2.111579

Se compararon las políticas schedule(static) y schedule(dynamic) en el ciclo principal del programa de integración por Monte Carlo. Ambas versiones se ejecutaron con (N=10^8) muestras y cuatro hilos, realizando cinco ejecuciones para cada política. Como medida representativa se utilizó la mediana de los tiempos registrados.Para schedule(static), los tiempos fueron 0.123143, 0.122071, 0.122361, 0.111281 y 0.119371 segundos, con una

mediana de 0.122071 segundos. Para schedule(dynamic), los tiempos fueron 2.131944, 2.043919, 2.113619, 2.087545 y 2.111579 segundos, con una mediana de 2.111579 segundos.

Los resultados muestran que la versión estática fue aproximadamente 17.3 veces más rápida que la dinámica en las condiciones evaluadas. Ambas versiones obtuvieron 78 538 673 aciertos y una estimación de π de 3.1415469200, por lo que la política de planificación no modificó el resultado numérico de estas pruebas.

Esta diferencia de rendimiento se explica porque las iteraciones del método Monte Carlo tienen un coste computacional aproximadamente uniforme. Por ello, la planificación estática permite distribuir el trabajo con una sobrecarga reducida. En cambio, la planificación dinámica, al utilizar por defecto bloques de una iteración cuando no se especifica un tamaño de bloque, puede introducir un coste elevado al asignar repetidamente nuevas iteraciones a los hilos.

Parte III. Evaluación de desempeño: speedup y ley de Amdahl

El desempeño de un programa paralelo se estudia con dos magnitudes. El speedup con p hilos es

$$S(p) = T(1)\ T(p)\ ,$$

donde T(p) es el tiempo de ejecución con p hilos, y la eficiencia es E(p) = S(p)/p. Si una fracción f del trabajo total es paralelizable ($0 \leq f \leq 1$) y el resto es inevitablemente secuencial, la ley de Amdahl establece el speedup máximo alcanzable:

$$S(p) = 1\ (1 - f) + f\ p\ .$$

Protocolo experimental

Con N = 108 fijo, mida T(p) para $p \in \{1, 2, 4, 8\}$ (ajuste el conjunto según los núcleos disponibles y las indicaciones de su docente). Para cada valor de p realice al menos cinco ejecuciones y registre la mediana de los tiempos; justifique por qué la mediana es más apropiada que el promedio en este experimento.

p (hilos)	Mediana de $T(p)$ [s]	$S(p) = T(1)/T(p)$	$E(p) = S(p)/p$
1		1,000	1,000
2			
4			
8			

```
========== HILOS: 1 ==========
--- Ejecucion 1 ---
Muestras: 100000000
Hilos solicitados: 1
Aciertos: 78538673
Estimacion de pi: 3.1415469200
Tiempo paralelo: 0.300347 segundos
--- Ejecucion 2 ---
Muestras: 100000000
Hilos solicitados: 1
Aciertos: 78538673
Estimacion de pi: 3.1415469200
Tiempo paralelo: 0.297210 segundos
--- Ejecucion 3 ---
Muestras: 100000000
Hilos solicitados: 1
Aciertos: 78538673
Estimacion de pi: 3.1415469200
Tiempo paralelo: 0.295754 segundos
--- Ejecucion 4 ---
Muestras: 100000000
Hilos solicitados: 1
Aciertos: 78538673
Estimacion de pi: 3.1415469200
Tiempo paralelo: 0.298859 segundos
--- Ejecucion 5 ---
Muestras: 100000000
Hilos solicitados: 1
Aciertos: 78538673
Estimacion de pi: 3.1415469200
Tiempo paralelo: 0.299123 segundos
```

```
========== HILOS: 2 ==========
--- Ejecucion 1 ---
Muestras: 100000000
Hilos solicitados: 2
Aciertos: 78538673
Estimacion de pi: 3.1415469200
Tiempo paralelo: 0.160955 segundos
--- Ejecucion 2 ---
Muestras: 100000000
Hilos solicitados: 2
Aciertos: 78538673
Estimacion de pi: 3.1415469200
Tiempo paralelo: 0.150406 segundos
--- Ejecucion 3 ---
Muestras: 100000000
Hilos solicitados: 2
Aciertos: 78538673
Estimacion de pi: 3.1415469200
Tiempo paralelo: 0.157109 segundos
--- Ejecucion 4 ---
Muestras: 100000000
Hilos solicitados: 2
Aciertos: 78538673
Estimacion de pi: 3.1415469200
Tiempo paralelo: 0.160093 segundos
--- Ejecucion 5 ---
Muestras: 100000000
Hilos solicitados: 2
Aciertos: 78538673
Estimacion de pi: 3.1415469200
Tiempo paralelo: 0.161444 segundos
```

```
========== HILOS: 4 ==========
--- Ejecucion 1 ---
Muestras: 100000000
Hilos solicitados: 4
Aciertos: 78538673
Estimacion de pi: 3.1415469200
Tiempo paralelo: 0.129353 segundos
--- Ejecucion 2 ---
Muestras: 100000000
Hilos solicitados: 4
Aciertos: 78538673
Estimacion de pi: 3.1415469200
Tiempo paralelo: 0.105158 segundos
--- Ejecucion 3 ---
Muestras: 100000000
Hilos solicitados: 4
Aciertos: 78538673
Estimacion de pi: 3.1415469200
Tiempo paralelo: 0.120881 segundos
--- Ejecucion 4 ---
Muestras: 100000000
Hilos solicitados: 4
Aciertos: 78538673
Estimacion de pi: 3.1415469200
Tiempo paralelo: 0.114814 segundos
--- Ejecucion 5 ---
Muestras: 100000000
Hilos solicitados: 4
Aciertos: 78538673
Estimacion de pi: 3.1415469200
Tiempo paralelo: 0.115829 segundos
```

```
========== HILOS: 8 ==========
--- Ejecucion 1 ---
Muestras: 100000000
Hilos solicitados: 8
Aciertos: 78538673
Estimacion de pi: 3.1415469200
Tiempo paralelo: 0.087701 segundos
--- Ejecucion 2 ---
Muestras: 100000000
Hilos solicitados: 8
Aciertos: 78538673
Estimacion de pi: 3.1415469200
Tiempo paralelo: 0.083797 segundos
--- Ejecucion 3 ---
Muestras: 100000000
Hilos solicitados: 8
Aciertos: 78538673
Estimacion de pi: 3.1415469200
Tiempo paralelo: 0.088844 segundos
--- Ejecucion 4 ---
Muestras: 100000000
Hilos solicitados: 8
Aciertos: 78538673
Estimacion de pi: 3.1415469200
Tiempo paralelo: 0.095944 segundos
--- Ejecucion 5 ---
Muestras: 100000000
Hilos solicitados: 8
Aciertos: 78538673
Estimacion de pi: 3.1415469200
Tiempo paralelo: 0.093697 segundos
```

p **(hilos)**	**Mediana de** $T(p)$ **[s]**	$S(p) = T(1)/T(p)$	$E(p) = S(p)/p$
1	**0,298859**	**1,000**	**1,000**
2	**0,160093**	**1,867**	**0,933**
4	**0,115829**	**2,580**	**0,645**
8	**0,088844**	**3,364**	**0,421**

Preguntas

III.1 Aplique la ley de Amdahl a los siguientes casos y interprete cada resultado:

(a) f = 0,98, p = 8;

$$S(8) = \frac{1}{(1 - 0,98) + \frac{0,98}{8}}$$

$$S(8) = \frac{1}{0,02 + 0,1225} = \frac{1}{0,1425}$$

$$\boxed{S(8) \approx 7,02}$$

Aunque el 98 % del programa sea paralelizable, el speedup no alcanza el valor ideal de 8. Esto se debe a que el 2 % restante sigue ejecutándose secuencialmente.

(b) f = 0,90, p = 8;

$$S(8) = \frac{1}{(1 - 0,90) + \frac{0,90}{8}}$$

$$S(8) = \frac{1}{0,10 + 0,1125} = \frac{1}{0,2125}$$

$$\boxed{S(8) \approx 4,71}$$

En este caso, el speedup es menor porque el 10 % del programa no puede paralelizar. Esto demuestra que una fracción secuencial más grande limita el aprovechamiento de los hilos.

(c) f = 0,98 y p creciendo sin límite.

$$S_{\max} = \frac{1}{1 - f} = \frac{1}{1 - 0,98}$$

$$\boxed{S_{\max} = 50}$$

¿Qué le enseña el caso (c) sobre la fracción secuencial de un programa?

Este resultado demuestra que agregar más hilos no produce una mejora ilimitada. Incluso con infinitos hilos, la fracción secuencial impone un límite al speedup máximo. Cuanto menor sea esa fracción, mayor será el rendimiento potencial del programa.

III.2 Con sus mediciones, estime la fracción secuencial f de su programa: despeje f de la ley de Amdahl a partir del speedup medido con p = 4 y muestre el procedimiento completo. Si lo desea, contraste su resultado con la métrica de Karp–Flatt, e = 1/S(p) − 1/p 1 − 1/p , y explique qué mide.

- Por ley de Amdahl :

$$S(p) = \frac{1}{(1-f) + \frac{f}{p}}$$

Utilizamos el speedup medido con cuatro hilos:

$$S(4) = \frac{T(1)}{T(4)} = \frac{0{,}298859}{0{,}115829} \approx 2,58017$$

- Ahora despejamos f.

Invertimos ambos lados de la ecuación:

$$\frac{1}{S(p)} = 1 - f + \frac{f}{p}$$

Agrupamos los términos que contienen f:

$$\frac{1}{S(p)} = 1 - f\left(1 - \frac{1}{p}\right)$$

Despejamos f:

$$f = \frac{1-\frac{1}{S(p)}}{1-\frac{1}{p}}$$

Sustituimos $S(4) = 2,58017$ y $p = 4$:

$$f = \frac{1-\frac{1}{2{,}58017}}{1-\frac{1}{4}}$$

$$f = \frac{1-0{,}38757}{0{,}75}$$

$$f = \frac{0{,}61243}{0{,}75}$$

$$f \approx 0.81657$$

La fracción paralelizable estimada es aproximadamente el 81,66 %. En consecuencia, la fracción secuencial estimada es:

$$1 - f = 1 - 0,81657$$

$$1 - f = 0.18343 = 18.34\,\%$$

Esta fracción secuencial es una estimación efectiva basada en las mediciones, no necesariamente el porcentaje exacto de instrucciones que no pueden paralelizar. Puede incluir efectos de sobrecarga y otros costes que no recoge el modelo ideal de Amdahl.

- **Contraste con la métrica de Karp–Flatt**

La métrica de Karp–Flatt se calcula mediante:

$$e = \frac{\frac{1}{S(p)} - \frac{1}{p}}{1 - \frac{1}{p}}$$

Sustituimos los datos de cuatro hilos:

$$e = \frac{\frac{1}{2,58017} - \frac{1}{4}}{1 - \frac{1}{4}}$$

$$e = \frac{0,38757 - 0,25}{0,75}$$

$$e \approx 0,18343$$

La métrica de Karp–Flatt estima la fracción secuencial efectiva a partir del speedup medido. En este caso coincide con $1 - f$, como se espera al despejar la ley de Amdahl con los mismos datos.

III.3 Compare su speedup medido con el ideal lineal S(p) = p y con el predicho por la ley de Amdahl usando su f estimado. Explique las discrepancias que observe. Considere en su análisis, entre otros factores, la creación y destrucción de hilos, el costo de la reducción, los límites de ancho de banda de memoria y la jerarquía de caché, y el efecto de la frecuencia variable del procesador y del hiperprocesamiento.

Para esta comparación utilizamos la fracción paralelizable estimada $f = 0,81657$. El modelo de Amdahl predice:

$$S(p) = \frac{1}{0{,}18343 + \frac{0{,}81657}{p}}$$

Hilos p	**Speedup ideal $S(p) = p$**	**Speedup medido**	**Predicción de Amdahl**
1	1,000	1,000	1,000
2	2,000	1,867	1,690
4	4,000	2,580	2,580
8	8,000	3,364	3,503

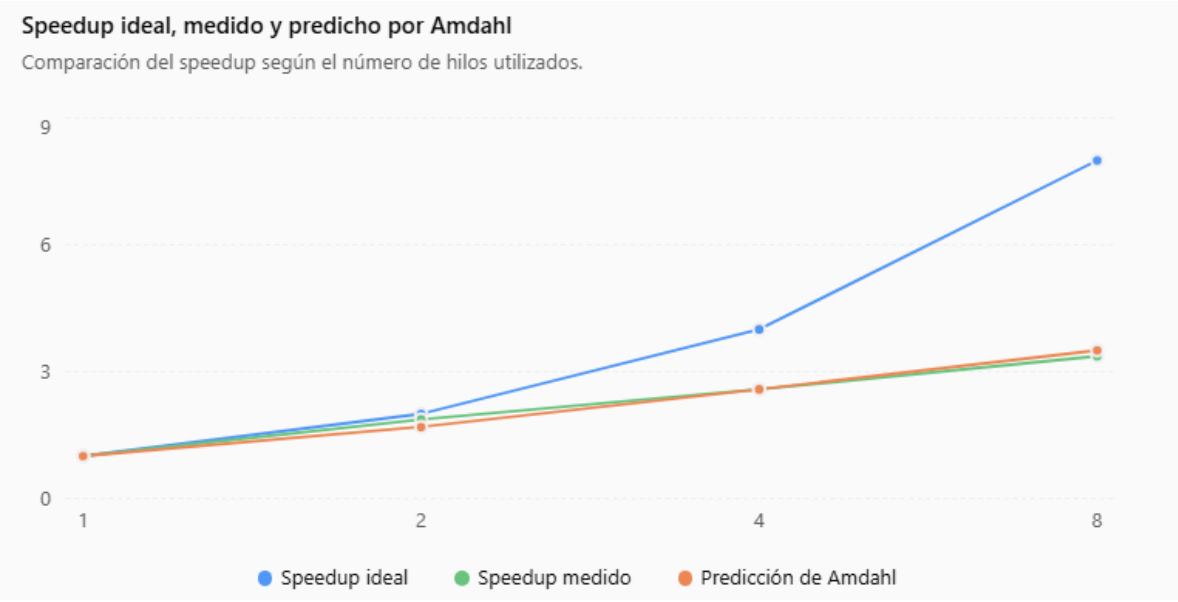


Figura 2. Comparación del speedup ideal, el medido experimentalmente y el predicho por la ley de Amdahl en función del número de hilos.

El modelo de Amdahl coincide con el resultado de cuatro hilos porque la fracción paralelizable se estimó precisamente a partir de esa medición.

Las diferencias entre el rendimiento ideal y los resultados obtenidos se pueden explicar por varios factores. En primer lugar, trabajar con varios hilos requiere coordinarlos para repartir las tareas y reunir los resultados, lo que introduce cierto tiempo adicional. También está el proceso de reducción de OpenMP, que debe sumar los aciertos encontrados por cada hilo.

Por otra parte, el acceso a la memoria y a la caché puede afectar el rendimiento, ya que los hilos comparten recursos del procesador. Además, la frecuencia del procesador puede variar según la carga de trabajo y la temperatura del equipo, haciendo que los tiempos cambien entre ejecuciones.

En nuestro caso, el computador tiene cuatro núcleos físicos y ocho hilos lógicos. Esto significa que, al utilizar ocho hilos, algunos deben compartir los recursos de un mismo núcleo, por lo que el rendimiento no aumenta en la misma proporción que la cantidad de hilos.

Finalmente, la ley de Amdahl ofrece una predicción teórica que no contempla todos estos factores. Como la fracción secuencial se estimó a partir de la medición con cuatro hilos, el modelo coincide con ese resultado, pero puede presentar diferencias al compararlo con las mediciones de dos y ocho hilos.

III.4 Suponga que le ofrecen tiempo de cómputo en una máquina con 64 núcleos. Con su fracción f estimada, ¿qué speedup máximo podría obtener y en qué se diferenciaría de crecer el número de muestras N? Reflexione: aumentar p con N fijo, ¿mejora la precisión del resultado o sólo el tiempo de cómputo? ¿Qué mejora cada una de las dos variables?

Utilizamos la fracción paralelizable estimada en III.2:

$$f \approx 0,81657$$

Su fracción secuencial es:

$$1 - f \approx 0,18343$$

Si el programa se ejecuta en una máquina con 64 núcleos, la ley de Amdahl predice:

$$S(64) = \frac{1}{(1-f)+\frac{f}{64}}$$

Sustituimos:

$$S(64) = \frac{1}{0{,}18343 + \frac{0{,}81657}{64}}$$

$$S(64) = \frac{1}{0{,}18343 + 0{,}01276}$$

$$S(64) = 5{,}10$$

Según esta estimación, el speedup sería aproximadamente de 5,10 veces. Aunque 64 núcleos representan una capacidad de procesamiento considerable, el porcentaje secuencial limita la mejora teórica.

La cantidad de hilos y el número de muestras cumplen funciones diferentes en el método de Monte Carlo. Si aumentamos el número de hilos p, el trabajo se reparte entre más unidades de procesamiento y, por lo general, se busca reducir el tiempo que tarda el programa en ejecutarse.

Por otro lado, si aumentamos el número de muestras N, obtenemos más puntos para realizar la estimación de π, lo que permite conseguir una mayor precisión. Sin embargo, también se necesitan más operaciones y, por tanto, más tiempo de cálculo si mantenemos la misma cantidad de hilos.

Esto se debe a que el error estadístico disminuye aproximadamente en proporción a:

$$\frac{1}{\sqrt{N}}$$

Por ejemplo, si aumentamos las muestras cien veces, el error típico se reduce aproximadamente diez veces.

Por tanto,aumentar los hilos ayuda principalmente a reducir el tiempo de ejecución, mientras que aumentar las muestras permite mejorar la precisión del resultado. Si mantenemos las mismas muestras y utilizamos exactamente los mismos puntos, agregar más hilos no cambia la estimación de πúnicamente permite calcularla en menos tiempo.

III.5 (Opcional.) Discuta brevemente por qué la ley de Amdahl puede resultar pesimista cuando, en lugar de fijar el tamaño del problema, lo que se fija es el tiempo total de cómputo (argumento de Gustafson).

La ley de Amdahl considera un tamaño de problema fijo y estudia cuánto puede acelerarse su ejecución al aumentar el número de procesadores. Bajo esta perspectiva, la fracción secuencial limita el speedup máximo. Sin embargo, en muchas aplicaciones paralelas se

dispone de más capacidad de procesamiento y se desea resolver un problema mayor en un tiempo similar. Este escenario se estudia mediante la ley de Gustafson, que expresa el speedup escalado como:

$$S_G(p) = p - \alpha(p - 1)$$

Donde α representa la fracción secuencial del tiempo de ejecución paralelo.

La diferencia es que, en lugar de mantener fijo el tamaño del problema, Gustafson considera que puede crecer la parte paralelizable del trabajo al aumentar los procesadores. Por eso, la ley de Amdahl puede resultar pesimista cuando el objetivo es aprovechar más recursos para resolver problemas más grandes en un tiempo limitado. En el método de Monte Carlo, por ejemplo, disponer de más núcleos podría permitir generar muchas más muestras en un tiempo similar, mejorando la precisión de la estimación de π.